

Beyond Tokens

A Unified Framework for Latent Communication in LLM-based Multi-Agent Systems

하서준

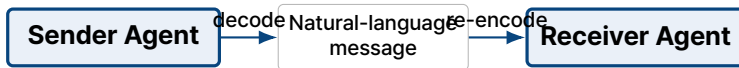
DGIST

2026년 7월 8일

Latent Communication · Hidden States · KV-Cache · Multi-Agent LLMs

Introduction: 왜 latent communication이 필요한가?

기존 LLM-based MAS의 기본 구조



Text-only communication의 세 가지 한계

1. High inference cost

모든 메시지에 sender는 token을 생성하고, receiver는 그 token을 다시 encoding해야 한다.

2. Information loss during discretization

고차원 hidden state가 vocabulary token 하나로 압축된다. 논문은 token의 정보량을 대략 15-17 bits로 보고, hidden state는 훨씬 큰 capacity를 가진다고 설명한다.

3. Redundancy and ambiguity of natural language

자연어는 사람이 읽기 좋지만 장황하고, hedge 표현이나 모호한 지시어 때문에 task-relevant density가 낮을 수 있다.

Natural language vs. Latent communication: 언제 무엇을 쓰나?

Latent communication이 natural language를 완전히 대체하는 것은 아니다.

Natural Language Communication

자연어를 사용해야 하는 경우

- ▶ 사람의 해석이 필요할 때
- ▶ 자연어 message는 사람이 즉시 읽을 수 있음
- ▶ debugging, alignment auditing, safety review에 유리
- ▶ human-AI interaction에서 그대로 설명하고 검토 가능

사용할 때

- ▶ final answer를 제공할 때
- ▶ 사용자에게 justification을 보여줘야 할 때
- ▶ human oversight / cross-organization interoperability 필요

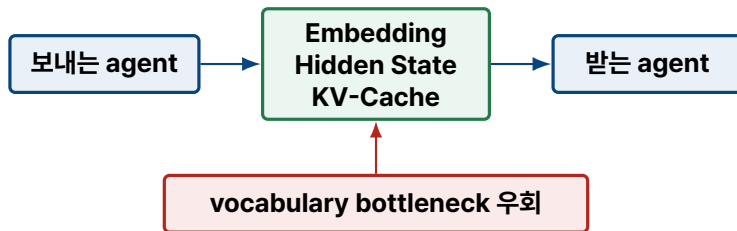
Latent Communication

Latent를 선호하는 경우

- ▶ 강하게 연결된 **agents**
planner → executor처럼 바로 이어지는 agents
- ▶ 중간 **communication**
사용자가 message를 볼 필요가 없는 단계
- ▶ 공유되거나 정렬된 **latent space**
same backbone 또는 compatible architectures
- ▶ **latency**가 핵심 제약인 경우
real-time pipeline, edge deployment, large agent counts

무엇이 달라질 수 있나?

핵심 아이디어: agent가 자연어를 생성하지 않고 내부 연속 표현을 직접 전달한다.



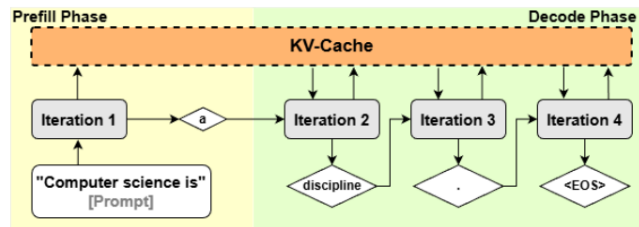
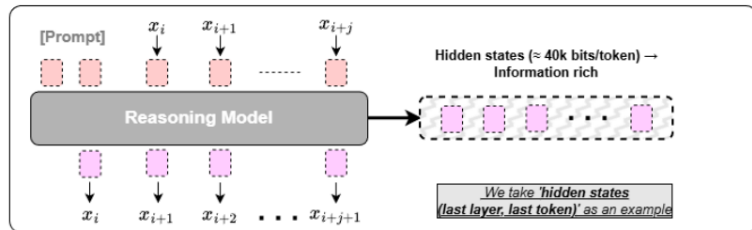
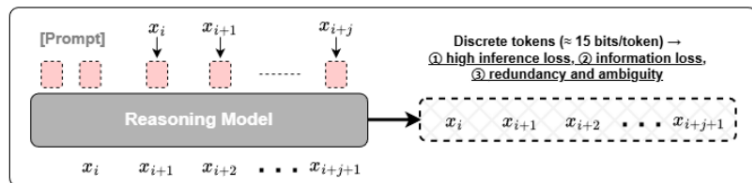
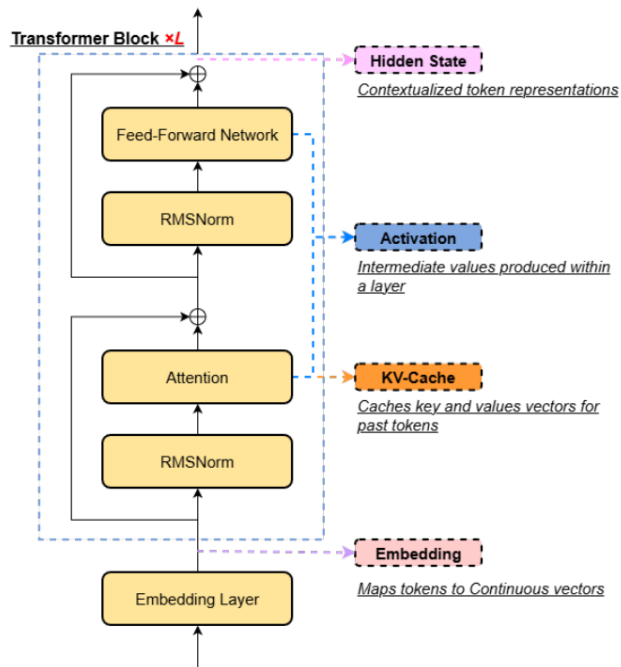
이 논문의 3-axis 분석 틀

WHAT
무엇을 보낼까?
Embedding / Hidden State / KV-Cache

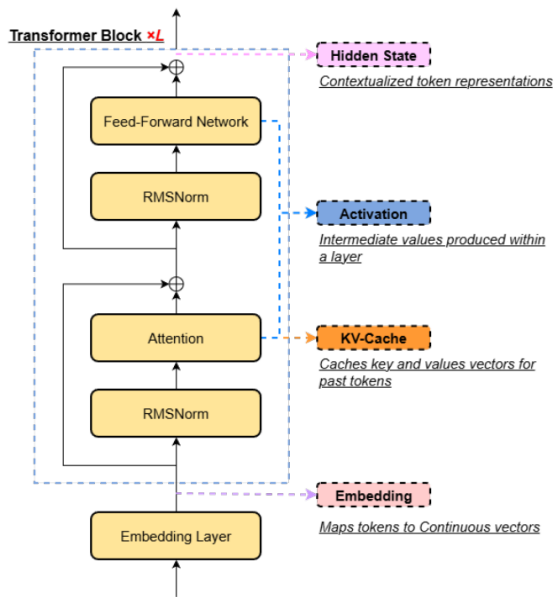
WHICH
어디에 맞출까?
layer / latent alignment

HOW
어떻게 합칠까?
concatenate / prepend / fuse

Method = ($\underbrace{\text{WHAT}}_{\text{type of information}}$, $\underbrace{\text{WHICH}}_{\text{alignment}}$, $\underbrace{\text{HOW}}_{\text{fusion}}$)



WHAT: 어떤 정보를 전달할까?



Embedding

- ▶ 입력 token x_i 를 dense vector로 바꾼 것
- ▶ 첫 Transformer block에 들어가는 가장 낮은 수준의 연속 표현

$$e_i \in \mathbb{R}^d$$

Hidden State

- ▶ token i 가 layer l 의 Transformer block을 지난 출력
- ▶ attention과 문맥이 반영된 layer-wise semantic representation

$$h_i^{(l)} \in \mathbb{R}^d$$

KV-Cache

- ▶ self-attention layer에서 token별 key/value를 저장한 것
- ▶ receiver가 sender context에 다시 attention할 수 있게 해주는 memory

$$KV = \{(k_i^{(l)}, v_i^{(l)})\}_{i=1, l=1}^{T, L}$$

WHICH: sender와 receiver를 어디서 맞출까?

두 종류의 alignment가 있다.

Latent information alignment

- ▶ 같은 model이면 latent space가 거의 동일하다.
- ▶ 같은 backbone의 fine-tune이면 projection head를 쓸 수 있다.
- ▶ heterogeneous architecture이면 universal codec, interaction layer 등 learned alignment가 필요하다.

Layer alignment

- ▶ **Last** → **First**: sender last layer를 receiver first layer로.
- ▶ **All** → **Corresponding**: sender layer l을 receiver layer l로.
- ▶ **Selected** → **Selected**: 중요한 일부 layer만 선택.

같은 backbone에서는 training-free alignment가 가능하지만, cross-architecture latent communication은 아직 어려운 open problem이다.

HOW: receiver computation에 어떻게 합칠까?

HOW	Mechanism	Trade-off
Concatenation	sender latent를 receiver prompt embedding 또는 hidden state token axis에 붙인다.	가장 단순하고 흔한 방식. hidden-state method에 잘 맞음.
Prepending	sender KV를 receiver KV-cache 앞에 붙인다. receiver가 sender context에 attention할 수 있다.	KV-cache method의 자연스러운 fusion. long context에서 유리.
Mathematical operation	addition, subtraction, linear projection, gated combination 등으로 합친다.	concat 보다 expressive 하지만 shape과 latent space compatibility가 필요.
Cross-attention	receiver가 sender latent set에 learned attention을 수행한다.	가장 expressive하지만 training이 필요. MoT 계열.
Cache restoration	sender KV-cache를 receiver cache로 직접 복원/교체해 recomputation을 피한다.	가장 효율적일 수 있지만 적용 범위가 제한적.

정리하면, latent communication 방법은 무엇을 보내고(WHAT), 어디에 맞추고(WHICH), 어떻게 합치는지(HOW)로 설명된다.

Empirical insights: 무엇이 잘 먹히나?

1. Long context일수록 이득이 큼

receiver가 긴 prompt를 다시 encoding하지 않아도 되기 때문.

2. Same-model agents가 지배적

대부분 sender와 receiver가 같은 backbone을 공유한다고 가정.

3. KV-cache가 emerging default

18개 방법 중 9개가 KV-cache를 communicated quantity로 사용.

핵심 trade-off: 정보량이 많을수록 latency는 줄기 쉽지만, payload와 architecture dependence는 커진다.